Kolla Integration

1. Overview

1.1 Description

Kolla Integration connects **Kolla** (unified dental API built on OpenDental EHR) with **Newo Platform**.

It enables:

- **Dynamic provider selection** based on treatment type (e.g., cleaning → Hygienist, filling → General Dentist)
- Checking real-time appointment availability based on provider schedule
- Booking dental appointments for both new and existing patients
- Cancelling existing appointments
- Automatic existing patient detection and appointment history retrieval during conversations
- Provider and operatory resource management

This integration is designed for **dental clinic administrators and front-desk teams** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a patient asks about available slots → Dynamically resolve provider by treatment type, check provider schedule, subtract booked appointments, return free slots
- When a patient wants to book an appointment → Resolve appropriate provider, detect new vs existing patient, create appointment
- When a patient wants to cancel an appointment → Cancel appointment in Kolla with provider reference
- When a patient's phone number is identified → Automatically look up existing contact and their upcoming appointments
- When a conversation ends → Clean up patient context from the agent's prompt

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	Provider schedule-based algorithm
Dynamic Provider Resolution	✅	LLM matches treatment type to provider taxonomy (feature-flagged)
Book Appointment (Existing)	✅	Uses existing contact record, marked "(EP)"
Book Appointment (New)	✅	Creates contact + appointment atomically, marked "(NP)"
Cancel Appointment	✅	Via provider-referenced cancellation
Existing Patient Lookup	✅	By phone number with auto-normalization
Appointment History	✅	Retrieves upcoming appointments for existing patients

Dynamic Provider Selection	✅	Feature-flagged. Based on service request + provider taxonomy or custom criteria (Gen())
Custom Provider Criteria	✅	Optional provider:service mapping for fine-tuned doctor selection
Default Provider Fallback	✅	Configurable fallback when resolution disabled or not possible
Operatory Selection	✅	Configurable default operatory/room
Configurable Slot Duration	✅	Default 30 min, configurable via attribute
Booking Window	✅	Configurable days ahead (default: 7)
Multi-Location Support	❌	Single operatory per instance
Reschedule Appointment	❌	Cancel + rebook as workaround

2. Requirements

Before installation:

- Active **Kolla** account with API access
- **Kolla API Token** (Bearer token)
- **Connector ID** (e.g., `opendental`)
- **Consumer ID** (e.g., `kolla-opendental-sandbox`)
- Kolla API base URL (default: `https://unify.kolla.dev/dental/v1/`)

3. How to Setup

3.1 Step 1 — Obtain Credentials from Kolla

1. Log in to your **Kolla** account
2. Navigate to the API / Integrations section
3. Obtain the following:
    - **API Token** (Bearer token for authentication)
    - **Connector ID** (identifies the dental system connector, e.g., `opendental`)
    - **Consumer ID** (identifies your consumer instance)
4. Store all credentials securely

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`kolla_api_token`	✅	Bearer token for Kolla API authentication
`kolla_connector_id`	✅	Connector identifier (e.g., `opendental`)

`kolla_consumer_id`	✅	Consumer identifier (e.g., `kolla-opendental-sandbox`)
`kolla_base_url`	❌	API base URL (default: `https://unify.kolla.dev/dental/v1/`)
`kolla_timezone`	❌	Timezone for slot display (default: `America/Chicago`)
`kolla_booking_window`	❌	Days ahead for availability search (default: `7`)
`kolla_appointment_length`	❌	Default appointment duration in minutes (default: `30`)
`kolla_availability_slot_offset`	❌	Days offset from current date (default: `0`)
`kolla_default_provider`	❌	Fallback provider when dynamic resolution unavailable (auto-populated)
`kolla_default_operatory`	❌	Selected operatory/room for bookings (auto-populated)
`kolla_providers_catalog`	❌	JSON catalog of providers with taxonomy (auto-populated)
`kolla_operatories_list`	❌	JSON list of operatory names for room availability check (auto-populated)
`kolla_slots_available`	❌	Enable availability checking (default: `True`)
`kolla_booking_available`	❌	Enable booking capability (default: `True`)
`kolla_cancelation_available`	❌	Enable cancellation capability (default: `True`)
`kolla_enable_provider_selection`	❌	Enable AI-based provider selection (default: `False`). When disabled, always uses default provider. When enabled, LLM matches service request to the best provider.
`kolla_selection_provider_criteria`	❌	Custom provider-to-service mapping for AI selection. Format: `ProviderName:service1; AnotherProvider:service2`. Example: `Jones, Tina:cleaning, hygiene; Albert, Brian:filling, root canal`. If empty, AI uses provider taxonomy from catalog.

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Validates the API credentials
    - Fetches available resources (providers and operatories) from Kolla
    - Populates provider and operatory selection dropdowns
    - Builds `kolla_providers_catalog` with taxonomy data for dynamic provider resolution
    - Builds `kolla_operatories_list` for room availability checking

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Kolla

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open slots
Book Appointment	`book_slot`	When the agent confirms a booking
Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Phone Number Identified	`define_user_phone_number`	When the patient's phone is collected mid-convo
Session Ended	`end_session`	Cleans up patient context from prompt
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Get Resources** — Fetches providers and operatories, builds providers catalog (GET `/resources`)
- **Search Contact** — Finds existing patient by phone (GET `/contacts`)
- **Get Appointments** — Retrieves appointments for a contact (GET `/appointments`)
- **Resolve Provider** — Dynamically selects provider based on `service_request` and provider taxonomy using inline `Gen()`
- **Load Provider Schedule** — Fetches resolved provider availability blocks (GET `/{provider}:loadSchedule`)
- **Get Existing Appointments** — Fetches booked appointments for date range (GET `/appointments`)
- **Create Appointment (Existing Patient)** — Books with resolved provider for known patient (POST `/appointments`)
- **Create Appointment (New Patient)** — Creates patient + appointment atomically with resolved provider (POST `/appointments:createNewPatientAppointment`)
- **Cancel Appointment** — Cancels an appointment (POST `/appointments/{id}:cancel`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Kolla Integration
    participant API as Kolla API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /resources
```

```
    API-->>I: Providers (with taxonomy) + operatories
    I->>I: Build providers_catalog for dynamic resolution
    I->>A: Resources configured

    Note over U,API: Patient Identified
    U->>A: (phone number detected)
    A->>I: define_user_phone_number (phone)
    I->>API: GET /contacts?filter=phone=(phone)
    API-->>I: Contact data
    I->>API: GET /appointments?filter=contact_id=(id)
    API-->>I: Existing appointments
    I->>A: Inject patient context into prompt

    Note over U,API: Availability Check
    U->>A: "I need a teeth cleaning this week"
    A->>I: get_available_slots (date, service_request)
    I->>I: Gen() resolves provider by service_request + taxonomy
    Note right of I: "teeth cleaning" → Hygienist → provider_2
    I->>API: GET /(resolved_provider):loadSchedule
    API-->>I: Provider schedule blocks
    I->>API: GET /appointments (date range, all providers)
    API-->>I: All appointments (with operatory info)
    I->>I: Subtract doctor appts, slice into slots, filter by room availability
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 10:00 AM please"
    A->>I: book_slot (customerInfo, dateTime, service_request)
    I->>I: Gen() resolves provider by service_request
    alt Existing patient
        I->>API: POST /appointments (contact_id, slot, resolved_provider)
    else New patient
        I->>API: POST /appointments:createNewPatientAppointment (contact +
appointment)
    end
    API-->>I: appointment_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: POST /appointments/(id):cancel (canceler)
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

KollaCheckAvailabilityFlow

```
flowchart TD
    E["get_available_slots<br>(date, service_request)"] --> GATE{"Slot
check<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| FF{"enable_provider<br>_selection?"}
    FF -->|"False"| DEFAULT["Use default_provider"]
    FF -->|"True"| RESOLVE{"service_request +<br>providers_catalog?"}
    RESOLVE -->|"Yes + criteria set"| GENCRIT["Gen() resolves provider<br>by custom
criteria"]
    RESOLVE -->|"Yes + no criteria"| GEN["Gen() resolves provider<br>by taxonomy
match"]
    RESOLVE -->|"No"| DEFAULT
    GEN --> PROV["GET /{resolved_provider}:loadSchedule<br>{date range}"]
    GENCRIT --> PROV
    DEFAULT --> PROV
    PROV --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| DATES{"Schedule<br>dates found?"}
    DATES -->|"No"| EMPTY["result_success<br>{availability: []}"]
    DATES -->|"Yes"| APPTS["GET /appointments<br>{date range, all providers}"]
    APPTS --> CALC["1. Subtract doctor's appointments<br>2. Slice into candidate
slots<br>3. Filter: keep slots where<br>at least 1 operatory is free"]
    CALC --> OUT["result_success<br>{availability[]}"]
```

KollaAppointmentFlow

```
flowchart TD
    E["book_slot<br>(customerInfo, dateTime,<br>service_request)"] -->
GATE{"Booking<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| FF{"enable_provider<br>_selection?"}
    FF -->|"False"| DEFAULT["Use default_provider"]
    FF -->|"True"| RESOLVE{"service_request +<br>providers_catalog?"}
    RESOLVE -->|"Yes"| GEN["Gen() resolves provider<br>by taxonomy or criteria"]
    RESOLVE -->|"No"| DEFAULT
    GEN --> EX["Extract customerInfo:<br>name, phone, email"]
    DEFAULT --> EX
    EX --> SEARCH["GET /contacts<br>?filter=phone={phone}"]
    SEARCH --> FOUND{"Contact<br>found?"}
    FOUND -->|"Yes"| EP["POST /appointments<br>{contact_id, slot,
<br>resolved_provider, operatory}<br>(EP)"]
    FOUND -->|"No"| NP["POST /appointments<br>:createNewPatientAppointment<br>
{contact + appointment}<br>(NP)"]
    EP --> OK{"HTTP 200?"}
    NP --> OK
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id}"]
```

KollaExistingClientFlow

```
flowchart TD
    E["define_user_phone_number"] --> NORM["Normalize phone:<br>remove +, strip
leading 1"]
    NORM --> SEARCH["GET /contacts<br>?filter=phone={phone}"]
    SEARCH --> FOUND{"Contact<br>found?"}
    FOUND -->|"No"| DONE["No action"]
    FOUND -->|"Yes"| STORE["Store contact in<br>persona attribute"]
    STORE --> INJECT["Inject ExistingCustomerInfo<br>into agent prompt"]
    INJECT --> APPTS["GET /appointments<br>?filter=contact_id={id}"]
    APPTS --> PARSE["Parse appointments:<br>datetime, duration,<br>description,
contact"]
    PARSE --> SAVE["Store in persona<br>attribute: bookings"]
```

KollaCancelationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| API["POST /appointments/{id}:cancel<br>{canceler: provider}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. Each flow also has a dedicated test skill triggered via `test_*` webhook events with sample data.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify logs show successful resource fetch and `kolla_providers_catalog` populated
2. **Dynamic Provider**: Ask for "teeth cleaning" — verify `resolved_provider` is a Hygienist (e.g., `provider_2`). Ask for "filling" — verify General (e.g., `provider_1`)
3. **Existing Client**: Start a conversation from a known phone number — verify patient context injected into agent prompt
4. **Availability**: Ask the agent "I need a cleaning this week" — verify slots are based on the resolved provider's schedule
5. **Booking (existing)**: Book with a known patient phone — verify appointment marked "(EP)" with correct provider
6. **Booking (new)**: Book with an unknown phone — verify contact + appointment created, marked "(NP)"
7. **Cancellation**: Cancel an appointment — verify status changed in Kolla

If no action occurs:

- Ensure `kolla_api_token` , `kolla_connector_id` , and `kolla_consumer_id` are all set correctly

- Verify `kolla_base_url` points to the correct Kolla API endpoint
- Confirm the integration was re-published after configuration changes
- Check that feature flags are enabled (`kolla_slots_available` , `kolla_booking_available` , `kolla_cancelation_available`)
- If provider selection is not working: verify `kolla_enable_provider_selection` is `True` and `kolla_providers_catalog` is populated
- If criteria-based selection is not working: verify `kolla_selection_provider_criteria` format — use semicolons between providers, colons between name and services
- Verify `kolla_default_provider` , `kolla_default_operatory` , `kolla_providers_catalog` , and `kolla_operatories_list` were populated during setup

> *Note: This integration performs real operations in Kolla/OpenDental. Appointments created or cancelled through the agent are reflected in the live system.*

5. FAQ

Q: Does the integration import historical appointments? A: No. Only appointments created after activation are managed. However, existing patient appointments are retrieved for context when a known phone number is detected.

Q: Can I connect multiple Kolla accounts? A: Each integration instance supports one set of API credentials (token, connector ID, consumer ID). For multiple accounts, create separate integration instances.

Q: How does the availability algorithm work? A: The integration: (1) resolves the appropriate provider based on treatment type via `Gen()` ; (2) fetches the provider's schedule blocks; (3) subtracts the provider's own booked appointments to get doctor-free slots; (4) slices into fixed-length candidate slots (default: 30 min); (5) for each candidate slot, checks if at least one operatory (room) is free by counting how many rooms have overlapping appointments — if busy rooms < total rooms, the slot is available. Operatory availability is determined from existing appointments, not from operatory schedules.

Q: How does dynamic provider selection work? A: Provider selection is controlled by the `kolla_enable_provider_selection` feature flag (default: off). When **disabled**, all appointments use `kolla_default_provider` — no AI matching. When **enabled**, the integration uses an inline LLM call (`Gen()`) to match the treatment description to a provider. There are two modes: (1) **Taxonomy mode** — if `kolla_selection_provider_criteria` is empty, the AI matches based on provider taxonomy from the catalog (e.g., "cleaning" → Hygienist, "filling" → General); (2) **Criteria mode** — if custom criteria are set (e.g., `Jones, Tina:cleaning, hygiene; Albert, Brian:filling`), the AI uses these explicit mappings for more precise control. In both cases, if resolution fails, it falls back to `kolla_default_provider` .

Q: How do I configure custom provider selection criteria? A: Set `kolla_selection_provider_criteria` with provider:service pairs separated by semicolons. Each entry maps a provider name to the services they handle. Examples: `Jones, Tina:cleaning, teeth whitening, hygiene; Albert, Brian:filling, extraction, root canal; Lexington, Sarah:exam, checkup, consultation` . Use natural language descriptions — the AI understands synonyms (e.g., "cleaning" matches "dental hygiene"). Make sure `kolla_enable_provider_selection` is set to `True` for criteria to take effect.

Q: What happens if a patient already exists in the system? A: When a phone number is identified during the conversation, the integration searches for existing contacts. If found, the patient's info is injected into the agent's prompt and the booking uses the existing contact record (marked "EP"). If not found, a new patient is created atomically with the appointment (marked "NP").

Q: What are the "(EP)" and "(NP)" markers? A: "(EP)" means Existing Patient — the appointment was created for a known contact. "(NP)" means New Patient — a new contact was created along with the appointment. These markers help clinic staff identify patient status.

Q: How is the phone number normalized? A: The leading `+` is removed, and if the number is 11 digits starting with `1` (US country code), the `1` is stripped. For example, `+12125551234` becomes `2125551234`.

Q: What are connector-id and consumer-id headers? A: These are Kolla-specific identifiers. `connector-id` identifies the dental system type (e.g., `opendental`), and `consumer-id` identifies your specific consumer instance within Kolla. Both are required for all API calls alongside the Bearer token.